

REST API

Обновлено 29 мая 2026, 00:42 Андрей Коновалов

General Conventions

- Base path: `/api/v1`.
- Payload format: JSON.
- Protected endpoints use a JWT Bearer access token.
- Responses are wrapped in a common envelope:

```
{
  "code": 200,
  "message": ["success"],
  "data": {},
  "error": null
}
```

- Swagger is available at `/api` if `APP_SWAGGER_ENABLED=true`.

Auth

POST `/api/v1/auth/register`

Creates a user and returns a JWT pair.

Request:

```
{
  "email": "user@example.com",
  "username": "player1",
  "password": "secret123"
}
```

Validation:

Field	Rule
email	valid email
username	string, 3..15 chars
password	string, min 6 chars

Response data:

```
{
  "accessToken": "jwt",
  "refreshToken": "jwt"
}
```

POST /api/v1/auth/login

Logs in with email/password.

Request:

```
{
  "email": "user@example.com",
  "password": "secret123"
}
```

Response data: JWT pair.

POST /api/v1/auth/refresh

Refreshes a JWT pair using a refresh token.

Auth: refresh token guard.

Request:

```
{
  "refreshToken": "jwt"
}
```

Response data: JWT pair.

DELETE /api/v1/auth/logout

Deletes the refresh token for the current fingerprint.

Auth: access token.

Response data:

```
true
```

Users

GET /api/v1/users/profile

Returns the current user's profile.

Auth: access token.

Response data:

```
{
  "id": "uuid",
  "email": "user@example.com",
  "username": "player1"
}
```

GET /api/v1/users/me/live-tournament

Returns an active tournament summary for app-level recovery.

Auth: access token.

Response data when an active tournament is found:

```
{
  "hasActiveTournament": true,
  "tournament": {
    "id": "uuid",
    "title": "Spring Cup",
    "status": "ACTIVE",
    "roundId": "uuid",
    "roundNumber": 2,
    "phase": "VOTING"
  }
}
```

Response data when no active tournament is found:

```
{
  "hasActiveTournament": false,
  "tournament": null
}
```

Tournaments

GET /api/v1/tournaments

Returns a paginated list of draft tournaments.

Auth: access token.

Query:

Field	Default	Rule
page	1	integer, min 1
limit	20	integer, 1..100

Response data:

```
{
  "items": [
    {
      "id": "uuid",
      "createdAt": "2026-05-13T12:00:00.000Z",
      "updatedAt": "2026-05-13T12:00:00.000Z",
      "title": "Spring Cup",
      "description": "Text prompt tournament",
      "visibility": "PUBLIC",
      "status": "DRAFT",
      "roundsCount": 3,
      "submissionDurationSeconds": 30,
      "voteDurationSeconds": 30,
      "ownerId": "uuid",
      "participantCount": 4
    }
  ],
  "totalCount": 1
}
```

POST /api/v1/tournaments

Creates a draft tournament. The owner is automatically added to participants.

Auth: access token.

Request:

```
{
  "title": "Spring Cup",
  "description": "Text prompt tournament",
  "visibility": "PRIVATE",
  "roundsCount": 3,
  "submissionDurationSeconds": 30,
  "voteDurationSeconds": 30
}
```

Validation:

Field	Rule
title	string
description	optional string
visibility	PUBLIC or PRIVATE
roundsCount	integer, 1..4
submissionDurationSeconds	one of 15 , 30 , 45
voteDurationSeconds	one of 15 , 30 , 45

Response data:

```
{
  "id": "uuid",
  "createdAt": "2026-05-13T12:00:00.000Z",
  "updatedAt": "2026-05-13T12:00:00.000Z",
  "title": "Spring Cup",
  "description": "Text prompt tournament",
  "visibility": "PRIVATE",
}
```

```
"roundsCount": 3,  
"submissionDurationSeconds": 30,  
"voteDurationSeconds": 30,  
"status": "DRAFT",  
"inviteToken": "uuid",  
"ownerId": "uuid"  
}
```

GET /api/v1/tournaments/:id

Returns draft tournament details.

Auth: access token.

Response data:

```
{  
  "id": "uuid",  
  "title": "Spring Cup",  
  "description": "Text prompt tournament",  
  "visibility": "PUBLIC",  
  "status": "DRAFT",  
  "roundsCount": 3,  
  "submissionDurationSeconds": 30,  
  "voteDurationSeconds": 30,  
  "ownerId": "uuid",  
  "participantCount": 4,  
  "participants": [  
    {  
      "userId": "uuid",  
      "cumulativeScore": 0  
    }  
  ]  
}
```

POST /api/v1/tournaments/:id/join

Adds the current user to participants.

Auth: access token.

Request for a public tournament:

```
{}
```

Request for a private tournament:

```
{  
  "inviteToken": "uuid"  
}
```

Repeated join for an existing participant is idempotent: the endpoint returns `true`.

Response data:

```
true
```

POST /api/v1/tournaments/:id/leave

Removes a participant from a draft tournament.

Auth: access token.

Restrictions:

- only `DRAFT` ;
- the owner cannot leave;
- if the participant row is not found, the endpoint returns `true`.

Response data:

```
true
```

POST /api/v1/tournaments/:id/cancel

Cancels a draft tournament.

Auth: access token.

Restrictions:

- owner only;
- only `DRAFT` .

Side effects:

- `status = CANCELLED` ;
- participant rows are deleted;
- `tournament:cancelled` is sent;
- participants are ejected from the [Socket.IO](#) room.

Response data:

```
true
```

POST /api/v1/tournaments/:id/start

Starts a draft tournament.

Auth: access token.

Restrictions:

- owner only;
- only `DRAFT` ;
- at least 4 participants;

- participants must not be taking part in other active tournaments.

Side effects:

- `status = ACTIVE` ;
- round 1 is created;
- a prompt is assigned;
- the submission deadline is scheduled;
- `tournament:started` and `round:created` are sent.

Response data:

```
true
```

GET /api/v1/tournaments/:id/full

Returns the current tournament snapshot.

Auth: access token.

Restriction: access is allowed only for participants.

Response data:

```
{
  "id": "uuid",
  "title": "Spring Cup",
  "description": "Text prompt tournament",
  "visibility": "PUBLIC",
  "status": "ACTIVE",
  "roundsCount": 3,
  "submissionDurationSeconds": 30,
  "voteDurationSeconds": 30,
  "ownerId": "uuid",
  "participants": [
    {
```

```
    "userId": "uuid",
    "cumulativeScore": 2
  }
],
"currentRound": {
  "id": "uuid",
  "number": 1,
  "phase": "SUBMISSION",
  "prompt": {
    "key": "unexpected_superpower",
    "type": "TEXT",
    "content": {
      "en": "Invent an unexpected superpower...",
      "ru": "Придумайте неожиданную суперспособность..."
    }
  }
},
"submissionDeadline": "2026-05-13T12:00:30.000Z",
"submissionClosedAt": null,
"votingDeadline": null
}
```

The snapshot is minimal: it does not contain the full history of rounds, submissions, votes, or final results.

Rounds

POST /api/v1/rounds/:roundId/submissions

Creates or updates the current user's submission.

Auth: access token.

Request:

```
{
  "content": "My answer"
```

```
}

```

Validation:

Field	Rule
content	string, 1..4000 chars

Restrictions:

- the current user must be a participant;
- the round must be in SUBMISSION ;
- submissionDeadline must not have expired;
- submissionClosedAt must be null .

Response data: entity TournamentRoundSubmissionEntity .

POST /api/v1/rounds/:roundId/votes

Creates or updates a manual vote for the currently revealed submission.

Auth: access token.

Request:

```
{
  "submissionId": "uuid",
  "value": "LIKE"
}

```

Validation:

Field	Rule

submissionId	UUID
value	LIKE or DISLIKE

Restrictions:

- the round must be in `VOTING` ;
- the voting step must be `OPEN` ;
- `submissionId` must match `currentVotingSubmissionId` ;
- `votingDeadline` must not have expired;
- the voter must be a participant;
- the voter cannot be the submission author.

Response data: entity `TournamentRoundVoteEntity` .